

DOCUMENTACIÓN TÉCNICA

MÓDULO PRODUCTO

Trabajo práctico final – Sistema de gestión de tienda

Instituto Superior de Formación Técnica N.º 179
Tecnicatura Superior en Análisis de Sistemas
Dr. Carlos Pellegrini

ÍNDICE

1. Introducción y responsabilidad ····· 3

2. Dependencias y ubicación en la arquitectura ····· 3

3. Representación de un producto ····· 3

4. Funcionamiento de crear_producto() ····· 3

5. Validaciones detalladas ····· 4

6. Integración con inventario y persistencia ····· 4

7. Ejemplo de ejecución y errores ····· 5

1. Introducción y responsabilidad

El módulo BACKEND/producto.py define el formato válido de un producto y concentra las reglas básicas de validación de sus datos. Su función principal es crear un diccionario consistente que pueda ser almacenado por inventario.py y persistido en SQLite.

Es un módulo pequeño y deliberadamente independiente: no administra listas, no ejecuta consultas SQL, no atiende solicitudes HTTP. Se ocupa exclusivamente de comprobar los datos y devolver una representación normalizada.

2. Dependencias y ubicación en la arquitectura

Elemento	Relación
math.isfinite	Única dependencia externa; permite verificar que un precio sea un número finito.
inventario.py	Importa crear_producto() y la utiliza antes de agregar, modificar o reconstruir productos.
BD.persistencia	No es importada por producto.py. Recibe posteriormente los datos validados desde inventario.py.
servidor.py	No importa directamente producto.py; utiliza inventario.py, que aplica las validaciones.

La dirección de dependencia es producto.py ← inventario.py: inventario depende de producto, pero producto no depende de inventario. Esto evita una dependencia circular y facilita reutilizar la validación desde distintos puntos del sistema.

3. Representación de un producto

La función devuelve un diccionario con cuatro atributos obligatorios. El ID identifica de manera única al producto; el nombre lo describe; cantidad_disponible representa el stock actual; y precio indica el valor unitario.

Campo	Tipo esperado	Regla
id_producto	int	Entero positivo; no se admite bool.
nombre	str	Texto no vacío después de quitar espacios externos.
cantidad_disponible	int	Entero mayor o igual a cero; no se admite bool.
precio	int o float	Número finito y no negativo; se normaliza a float.

```
{
  "id_producto": 12,
  "nombre": "Teclado",
  "cantidad_disponible": 8,
  "precio": 15000.0
}
```

4. Funcionamiento de crear_producto()

crear_producto(id_producto, nombre, cantidad_disponible, precio) recibe los cuatro valores, valida cada uno en un orden definido y devuelve un diccionario nuevo. Si alguno incumple las reglas, lanza ValueError y no devuelve un producto parcial.

1. Valida el identificador: exige que type(id_producto) sea int y que el valor sea mayor que cero.
2. Valida el nombre: exige una cadena de texto y que, luego de aplicar strip(), no quede vacía.

3. Valida el stock: exige que type(cantidad_disponible) sea int y que el valor no sea negativo.
4. Valida el precio: exige un int o float, intenta convertirlo a float y comprueba que sea finito y no negativo.
5. Construye y devuelve el diccionario con el nombre sin espacios externos y el precio expresado como float.

5. Validaciones detalladas

5.1 Identificador entero positivo

La condición utiliza type(valor) is int, en lugar de isinstance(valor, int). La diferencia importa porque en Python bool es una subclase de int: True podría aceptarse como 1 si se utilizara isinstance(). El código evita ese caso y exige un entero real.

5.2 Nombre no vacío

Primero comprueba que nombre sea una cadena. Luego aplica strip(), que elimina espacios al principio y al final. Una cadena compuesta solamente por espacios se rechaza. El valor normalizado es el que finalmente se guarda en el diccionario.

5.3 Stock entero no negativo

La cantidad disponible debe ser un entero y puede ser cero, porque el sistema necesita representar productos agotados para permitir reservas o esperas. No puede ser negativa. También se excluyen valores booleanos mediante la comparación exacta del tipo.

5.4 Precio numérico, finito y no negativo

Solo se aceptan valores cuyo tipo exacto sea int o float. Luego se intenta convertir el precio a float. Si el número queda fuera del rango representable y se produce OverflowError, la función lo transforma en un ValueError con un mensaje entendible.

isfinite() rechaza valores especiales como infinito positivo, infinito negativo y NaN. Finalmente, el precio debe ser mayor o igual a cero. Un precio entero, por ejemplo 15000, se devuelve como 15000.0.

6. Integración con inventario y persistencia

inventario.py importa crear_producto() y lo invoca en tres situaciones centrales: al cargar los productos desde la base, al agregar un producto nuevo y al modificar uno existente. De esta forma, los datos que llegan a la lista enlazada mantienen siempre el mismo formato.

Al agregar, inventario utiliza un ID provisional para validar el resto de los campos. Después persiste los datos y reemplaza ese ID por el que genera SQLite. Al modificar, valida el nuevo conjunto de datos antes de tocar la base o la lista. Al cargar, vuelve a validar lo que ya estaba almacenado.

producto.py no conoce SQLite. Esta decisión separa la regla de negocio —qué datos constituyen un producto válido— del mecanismo de almacenamiento —cómo se guarda ese producto—.

```
servidor.py
   │ recibe los datos HTTP
   ▼
inventario.py
   │ llama a crear_producto()
   ▼
producto.py
   │ valida y normaliza
   ▼
inventario.py ──► persistencia.py (SQLite)
   │
   └──────────────► tad_listas.py (lista enlazada)
```

7. Ejemplo de ejecución y errores

7.1 Caso válido

```
crear_producto(12, "  Teclado  ", 8, 15000)

Resultado:
{"id_producto": 12, "nombre": "Teclado",
 "cantidad_disponible": 8, "precio": 15000.0}
```

El nombre queda sin espacios externos y el precio queda convertido a float. El stock y el ID conservan su tipo entero.

7.2 Casos rechazados

Entrada problemática	Motivo del rechazo
id_producto = 0	El ID debe ser positivo.
nombre = ' '	El nombre queda vacío después de strip().
cantidad_disponible = -2	El stock no puede ser negativo.
cantidad_disponible = True	bool no se acepta como entero de stock.
precio = float('inf')	El precio debe ser finito.
precio = -10	El precio no puede ser negativo.
precio = '1500'	El precio debe llegar como int o float, no como texto.

Los errores se comunican mediante ValueError. El módulo que llama a crear_producto() puede capturar esa excepción y traducirla a una respuesta adecuada; por ejemplo, servidor.py devuelve un error HTTP 400 cuando los datos enviados por el usuario no cumplen las reglas.